

DOCUMENTACIÓN API REST - PRODUCTOS

1. Introducción

Este documento describe el funcionamiento de una API REST desarrollada en PHP que permite gestionar productos almacenados en una base de datos MySQL.

La API devuelve y recibe información en formato JSON y está diseñada para ser reutilizable por cualquier frontend compatible con HTTP.

2. Tecnologías utilizadas

- Servidor web: Apache (XAMPP)
- Lenguaje de servidor: PHP
- Base de datos: MySQL
- Acceso a datos: PDO
- Formato de datos: JSON
- Protocolo: HTTP
- Cliente: HTML + JavaScript (fetch)

3. Base de datos

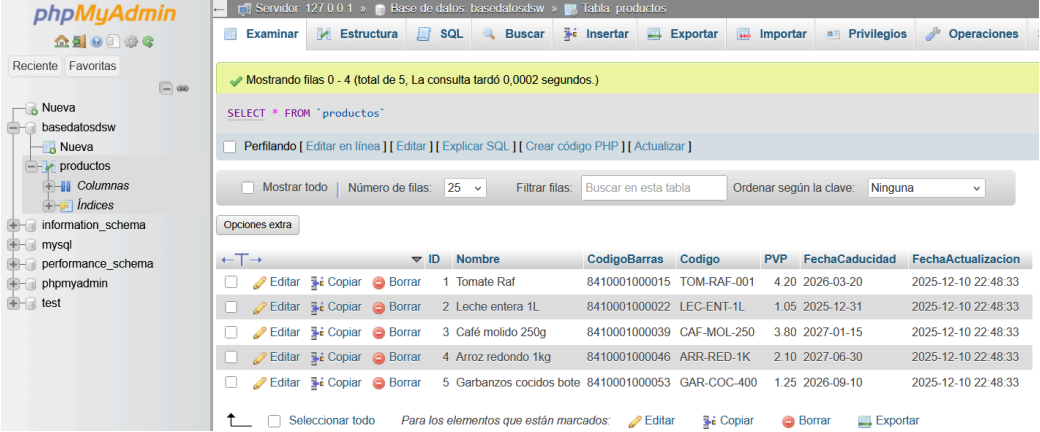
Base de datos: basedatosdsw

Tabla: productos

Campos principales:

- ID (clave primaria)
- Nombre
- CodigoBarras
- Codigo
- PVP
- FechaCaducidad
- FechaActualizacion

En esta captura se muestra la base de datos basedatosdsw con la tabla productos, donde se almacenan los datos gestionados por el servicio web.



Mostrando filas 0 - 4 (total de 5, La consulta tardó 0,0002 segundos)

SELECT * FROM `productos`

Perfilando [Editar en línea] [Editar] [Explicar SQL] [Crear código PHP] [Actualizar]

Mostrar todo | Número de filas: 25 | Filtrar filas: Buscar en esta tabla | Ordenar según la clave: Ninguna

Opciones extra

	ID	Nombre	CodigoBarras	Codigo	PVP	FechaCaducidad	FechaActualizacion
☐	1	Tomate Raf	8410001000015	TOM-RAF-001	4.20	2026-03-20	2025-12-10 22:48:33
☐	2	Leche entera 1L	8410001000022	LEC-ENT-1L	1.05	2025-12-31	2025-12-10 22:48:33
☐	3	Café molido 250g	8410001000039	CAF-MOL-250	3.80	2027-01-15	2025-12-10 22:48:33
☐	4	Arroz redondo 1kg	8410001000046	ARR-RED-1K	2.10	2027-06-30	2025-12-10 22:48:33
☐	5	Garbanzos cocidos bote	8410001000053	GAR-COC-400	1.25	2026-09-10	2025-12-10 22:48:33

Seleccionar todo | Para los elementos que están marcados: Editar Copiar Borrar Exportar

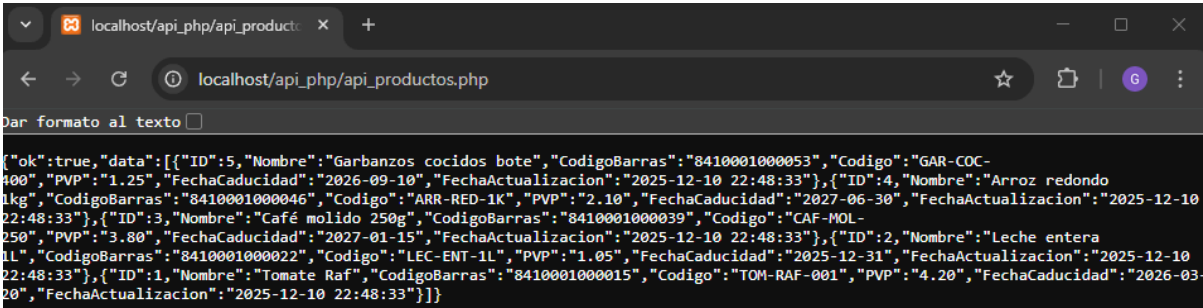
4. Endpoint principal

URL base:

http://localhost/api_php/api_productos.php

Todas las operaciones se realizan sobre este endpoint usando distintos métodos HTTP.

Respuesta del endpoint api_productos.php devolviendo el listado de productos en formato JSON mediante una petición HTTP GET.



localhost/api_php/api_productos.php

Dar formato al texto

```
{
  "ok": true,
  "data": [
    {
      "ID": 5,
      "Nombre": "Garbanzos cocidos bote",
      "CodigoBarras": "8410001000053",
      "Codigo": "GAR-COC-400",
      "PVP": "1.25",
      "FechaCaducidad": "2026-09-10",
      "FechaActualizacion": "2025-12-10 22:48:33"
    },
    {
      "ID": 4,
      "Nombre": "Arroz redondo 1kg",
      "CodigoBarras": "8410001000046",
      "Codigo": "ARR-RED-1K",
      "PVP": "2.10",
      "FechaCaducidad": "2027-06-30",
      "FechaActualizacion": "2025-12-10 22:48:33"
    },
    {
      "ID": 3,
      "Nombre": "Café molido 250g",
      "CodigoBarras": "8410001000039",
      "Codigo": "CAF-MOL-250",
      "PVP": "3.80",
      "FechaCaducidad": "2027-01-15",
      "FechaActualizacion": "2025-12-10 22:48:33"
    },
    {
      "ID": 2,
      "Nombre": "Leche entera 1L",
      "CodigoBarras": "8410001000022",
      "Codigo": "LEC-ENT-1L",
      "PVP": "1.05",
      "FechaCaducidad": "2025-12-31",
      "FechaActualizacion": "2025-12-10 22:48:33"
    },
    {
      "ID": 1,
      "Nombre": "Tomate Raf",
      "CodigoBarras": "8410001000015",
      "Codigo": "TOM-RAF-001",
      "PVP": "4.20",
      "FechaCaducidad": "2026-03-20",
      "FechaActualizacion": "2025-12-10 22:48:33"
    }
  ]
}
```

5. Operaciones disponibles (CRUD)

5.1 Obtener listado de productos

Método: GET

URL:

/api_productos.php

Respuesta correcta (200):

```
{
  "ok": true,
  "data": [ ... ]
}
```

5.2 Obtener producto por ID

Método: GET

URL:

/api_productos.php?id=3

Errores posibles:

404 → producto no encontrado

5.3 Crear producto

Método: POST

Body (JSON):

```
{
  "Nombre": "Pan",
  "PVP": 1.25
}
```

Respuesta correcta (201):

```
{
  "ok": true,
  "message": "Producto creado",
  "id": 6
}
```

5.4 Actualizar producto

Método: PUT

Body (JSON):

```
{
  "ID": 3,
  "Nombre": "Pan integral",
  "PVP": 1.40
}
```

Errores posibles:

- 400 → faltan campos
- 404 → producto no encontrado

5.5 Eliminar producto

Método: DELETE

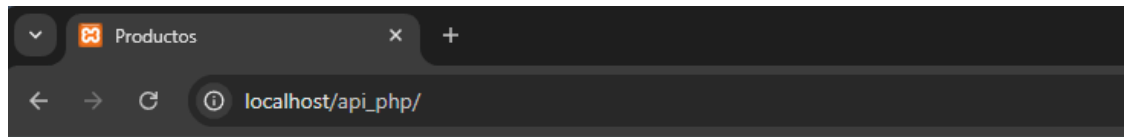
URL:

/api_productos.php?id=3

Respuesta correcta (200):

```
{
  "ok": true,
  "message": "Producto borrado"
}
```

Frontend desarrollado en HTML y JavaScript que consume la API REST mediante fetch() y muestra los productos obtenidos del servidor.



Listado de productos

✅ Productos cargados

ID	Nombre	Código barras	Código interno	PVP	Fecha caducidad	Actualizado
5	Garbanzos cocidos bote	8410001000053	GAR-COC-400	1.25	2026-09-10	2025-12-10 22:48:33
4	Arroz redondo 1kg	8410001000046	ARR-RED-1K	2.10	2027-06-30	2025-12-10 22:48:33
3	Café molido 250g	8410001000039	CAF-MOL-250	3.80	2027-01-15	2025-12-10 22:48:33
2	Leche entera 1L	8410001000022	LEC-ENT-1L	1.05	2025-12-31	2025-12-10 22:48:33
1	Tomate Raf	8410001000015	TOM-RAF-001	4.20	2026-03-20	2025-12-10 22:48:33

6. Control de errores y excepciones

La API controla distintos tipos de errores y responde siempre en formato JSON.

Ejemplos:

Error de conexión con la base de datos

```
{
  "ok": false,
  "error": "No hay conexión con la base de datos",
  "code": "DB_ERROR"
}
```

Error por datos incorrectos

```
{
  "ok": false,
  "error": "Faltan campos obligatorios",
  "code": "MISSING_FIELDS"
}
```

Error por ruta o uso incorrecto

```
{
  "ok": false,
  "error": "Respuesta no JSON",
}
```

```
"code": "NON_JSON"
}
```

Estos errores permiten que el frontend muestre mensajes claros al usuario final.

Productos

Front de prueba (otro alumno)

127.0.0.1:5500/DesarrolloWebEntornoServidor/api_php/front_prueba_otro.html

Front de prueba — API Productos

Esta página está pensada para simular el "front de otro alumno".

URL del endpoint

GET / productos

Probar CORS

✖ GET falló: Respuesta no-JSON

POST — Crear producto

Nombre *

Ej: Pan

PVP *

Ej: 1.25

Código de barras

Ej: 841...

Código

Ej: PAN-001

Fecha caducidad (YYYY-MM-DD)

2028-05-01

Crear (POST)

PUT — Actualizar producto

ID *

Ej: 3

Nombre *

Ej: Pan integral

PVP *

Ej: 1.40

Código de barras

Ej: 841...

Código

Ej: PAN-002

Fecha caducidad (YYYY-MM-DD)

2028-05-10

Actualizar (PUT)

DELETE — Borrar producto

ID *

Ej: 3

Borrar (DELETE) Nota: aquí usamos ?id=... en la URL, que es lo más simple.

GET — Producto por ID

ID *

Ej: 1

GET por ID

Respuesta

```
{
  "ok": false,
  "error": "Respuesta no-JSON",
  "code": "NON_JSON"
}
```

7. Control de acceso (CORS)

La API incluye configuración CORS para permitir el acceso desde otros frontends ubicados en distintos puertos u orígenes.

Esto garantiza que el servicio puede ser reutilizado por aplicaciones externas sin bloqueos del navegador.

Se ha verificado el correcto funcionamiento mediante un frontend de prueba externo y el botón “Probar CORS”.

“El botón ‘Probar CORS’ sirve para comprobar que la API puede ser consumida desde otros frontends en distintos puertos u orígenes, algo necesario para que el servicio sea reutilizable.”

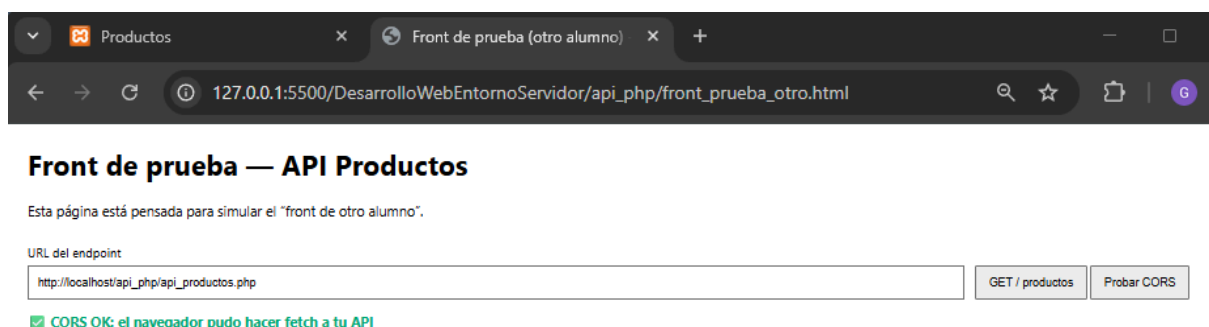
Cuando pulsas el botón:

1. El navegador intenta hacer un `fetch()` a la API.
2. Antes de permitir la respuesta, el navegador comprueba si el servidor **autoriza accesos externos**.
3. La API responde con estos headers:

Access-Control-Allow-Origin: *

Access-Control-Allow-Methods: GET, POST, PUT, DELETE, OPTIONS
4. El navegador entiende:

“Este servidor permite que cualquier frontend acceda”
5. La petición se completa correctamente y el botón muestra **CORS OK**.



Frontend independiente que simula el uso del servicio web por parte de otro alumno, comprobando que la API puede ser consumida desde otro cliente distinto.

Productos

Front de prueba (otro alumno)

127.0.0.1:5500/DesarrolloWebEntornoServidor/api_php/front_prueba_otro.html

Front de prueba — API Productos

Esta página está pensada para simular el "front de otro alumno".

URL del endpoint

http://localhost/api_php/api_productos.php

GET / productos

Probar CORS

✓ CORS OK: el navegador pudo hacer fetch a tu API

POST — Crear producto

Nombre *

Ej: Pan

PVP *

Ej: 1.25

Código de barras

Ej: 841...

Código

Ej: PAN-001

Fecha caducidad (YYYY-MM-DD)

2026-05-01

Crear (POST)

PUT — Actualizar producto

ID *

Ej: 3

Nombre *

Ej: Pan integral

PVP *

Ej: 1.40

Código de barras

Ej: 841...

Código

Ej: PAN-002

Fecha caducidad (YYYY-MM-DD)

2026-05-10

Actualizar (PUT)

DELETE — Borrar producto

ID *

Ej: 3

Borrar (DELETE) Nota: aquí usamos ?id=... en la URL, que es lo más simple.

GET — Producto por ID

ID *

Ej: 1

GET por ID

Respuesta

```
{
  "ok": true,
  "data": [
    {
      "ID": 5,
      "Nombre": "Garbanzos cocidos bote",
      "CodigoBarras": "8410001000053",
      "Codigo": "GAR-COC-400",
      "PVP": "1.25",
      "FechaCaducidad": "2026-09-10",
      "FechaActualizacion": "2025-12-10 22:48:33"
    },
    {
      "ID": 4,
      "Nombre": "Arroz redondo 1kg",
      "CodigoBarras": "8410001000046",
      "Codigo": "ARR-RED-1K",
      "PVP": "2.10",
      "FechaCaducidad": "2027-06-30",
      "FechaActualizacion": "2025-12-10 22:48:33"
    }
  ]
}
```

8. Conclusión

La API desarrollada es funcional, reutilizable y accesible mediante HTTP. Permite gestionar productos de forma segura y controlada, cumpliendo los requisitos del módulo de Entorno Servidor.